Python: exercises on strings and lists

This notebook was generated in pseudocode mode.

Exercise 1: Converting String to List

Exercise Description

Convert the string `s = "Python is fun!"` to a list of characters `L`.

Your solution

```
# Step 1: Define the string s
# Step 2: Convert string to list using list() function
# Step 3: Print the result
```

Test your solution

```
# Test the string to list conversion
expected = ['P', 'y', 't', 'h', 'o', 'n', ' ', 'i', 's', ' ', 'f', 'u', 'n', '!']
print("✓ Test passed: String converted to list correctly")
print(f"Result: {L}")
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 2: Splitting a String by Whitespace

Exercise Description

Split the string `s = "I love programming"` into a list of words.

Store the result in `L`.

Your solution

```
# Step 1: Define the string s
# Step 2: Split string by whitespace using split()
# Step 3: Print the result
```

Test your solution

```
# Test string splitting by whitespace
expected = ['I', 'love', 'programming']
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 3: Custom Character Split

Exercise Description

Split the string `s = "apple-banana-orange"` using `-` as the delimiter.

Store the result in `L`.

Your solution

```
# Step 1: Define the string s
# Step 2: Split string using '-' as delimiter
# Step 3: Print the result
```

Test your solution

```
# Test string splitting by custom delimiter
expected = ['apple', 'banana', 'orange']
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 4: Joining a List into a String

Exercise Description

Join the list `L = ['Hello', 'World']` into a string without spaces.

Your solution

```
# Step 1: Define the list L
# Step 2: Join list elements without spaces using join()
# Step 3: Print the result
```

Test your solution

```
# Test joining list without spaces
expected = 'HelloWorld'
assert s == expected, f"Expected {expected}, got {s}"
print("✓ Test passed")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 5: Joining with a Custom Character

Exercise Description

Join the list `L = ['data', 'science']` into a string `s` with an underscore `_` between words.

Your solution

```python
# Step 1: Define the list L
# Step 2: Join list elements with underscore using join()
# Step 3: Print the result
```

Test your solution

```python
# Test joining list with underscore
expected = 'data_science'
assert s == expected, f"Expected {expected}, got {s}"
print("✓ Test passed")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 6: Sorting a List

Exercise Description

Sort the list `L = [8, 3, 5, 2]` in ascending order using `sort()`.

Your solution

```
# Step 1: Define the list L
# Step 2: Sort the list in place using sort()
# Step 3: Print the sorted list
```

Test your solution

```
# Test list sorting in place
expected = [2, 3, 5, 8]
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed: List sorted correctly")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 7: Reversing a List

Exercise Description

Reverse the list `L = [1, 2, 3, 4]`.

Your solution

```
# Step 1: Define the list L
# Step 2: Reverse the list in place using reverse()
# Step 3: Print the reversed list
```

Test your solution

```
# Test list reversal in place
expected = [4, 3, 2, 1]
```

```
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 8: Using `sorted()` to Sort a List

Exercise Description

Use `sorted()` to sort the list `L = [10, 5, 8, 3]` without modifying the original list.

Store the result in `sorted_L`.

Your solution

```
# Step 1: Define the list L
# Step 2: Create new sorted list using sorted()
# Step 3: Print both original and sorted lists
```

Test your solution

```
# Test sorted() function
original = [10, 5, 8, 3]
expected_sorted = [3, 5, 8, 10]
assert sorted_L == expected_sorted, f"Expected {expected_sorted}, got {sorted_L}"
assert L == original, f"Original list should be unchanged, got {L}"
print("✓ Test passed: sorted() works correctly")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 9: Aliasing a List

Exercise Description

Create a list `a = [1, 2, 3]`, and assign `b = a`. Set the first element of `b` to `99` and show how it affects `a`.

Your solution

```
# Step 1: Create list a
# Step 2: Create alias b = a
# Step 3: Modify b and show effect on a
```

Test your solution

```
assert a == b
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 10: Cloning a List

Exercise Description

Clone the list `a = [4, 5, 6]` to a new list `b` and set the first element of `b` to `10` without affecting `a`.

Your solution

```
# Step 1: Create list a
# Step 2: Clone list to b using copy() or slicing
# Step 3: Modify b and show a is unchanged
```

Test your solution

```
assert a == [4, 5, 6]
assert b == [10, 5, 6]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 11: Nested Lists and Aliasing

Exercise Description

Create a nested list `colors = [warm]`, where `warm = ['yellow', 'orange']`. Add `hot = ['red']` to the nested list. Show how appending to `hot` affects `colors`.

Your solution

```
# Step 1: Create warm list
# Step 2: Create colors nested list
# Step 3: Create hot list and add to colors
# Step 4: Modify hot and show effect on colors
```

Test your solution

```
assert colors == [['yellow', 'orange'], ['red']]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 12: Mutating a List While Iterating

Exercise Description

Explain why the following code is problematic. Then, suggest a fix.

Your solution

```
# Step 1: Show the problematic code
# Step 2: Explain why it's problematic
# Step 3: Provide a fixed version
```

Test your solution

```
assert remove_dups([1, 2], [1, 2]) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 13: Removing Duplicates

Exercise Description

Write a function `remove_dups(L1, L2)` that removes all elements from `L1` that are also in `L2`.

Your solution

```python
def remove_dups(L1, L2):
    # Step 1: Iterate through L2 to find elements to remove
    # Step 2: For each element in L2, remove all occurrences from L1
    # Step 3: Handle list mutation properly during iteration
```

Test your solution

```python
# Test remove_dups function
L1 = [1, 2, 3, 4, 5]
L2 = [2, 4]
remove_dups(L1, L2)
expected = [1, 3, 5]
assert L1 == expected, f"Expected {expected}, got {L1}"
print("✓ Test passed: Duplicates removed correctly")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 14: List Mutation and Sorting

Exercise Description

Explain the difference between using `sort()` and `sorted()` on the list `L = [5, 3, 8]`.

Store the result in `sorted_L`.

Your solution

```
# Step 1: Create list L
# Step 2: Demonstrate sort() method (in-place)
# Step 3: Demonstrate sorted() function (returns new list)
# Step 4: Show the difference
```

Test your solution

```
assert L == [3, 5, 8]
assert sorted_L == [3, 5, 8]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 15: Using List Methods in a Loop

Exercise Description

Use a loop to sort and reverse the list `L = [7, 2, 9]`. Apply the transformations two times.

Your solution

```
# Step 1: Create list L
# Step 2: Use loop to apply sort() and reverse() twice
# Step 3: Print result after each transformation
```

Test your solution

```
assert L == [9, 7, 2]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 16: Reconstruct String from Substrings

Exercise Description

You are given a string `s = "data_science_is_fun"`. Split it into individual words and then reconstruct the string by joining the words with a hyphen `-`, but in reverse order.

Your solution

```
# Step 1: Define string s
# Step 2: Split string into words
# Step 3: Reverse the word order
# Step 4: Join with hyphens
```

Test your solution

```
assert words == ['data', 'science', 'is', 'fun']
assert reversed_s == 'fun-is-science-data'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 17: Alternating Split and Join

Exercise Description

Given the string `s = "The_quick_brown_fox"`, split the string at underscores `_` and then join the resulting list using spaces `' '`. Next, reverse the string and split it by spaces again.

Store the result in `split_s`.

Your solution

```
# Step 1: Define string s
# Step 2: Split by underscores
# Step 3: Join with spaces
# Step 4: Reverse and split by spaces again
```

Test your solution

```
assert split_s == ['The', 'quick', 'brown', 'fox']
assert joined_s == 'The quick brown fox'
assert reversed_s == 'xof nworb kciuq ehT'
assert reversed_split == ['xof', 'nworb', 'kciuq', 'ehT']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 18: List Mutation Inside a Function

Exercise Description

Write a function `append_and_sort(L)` that takes a list `L`, appends a given value (e.g., 10), sorts the list, and returns it. Make sure the original list is not mutated inside the function.

Your solution

```python
def append_and_sort(L):
    # Step 1: Create a copy of the list to avoid mutating original
    # Step 2: Append the given value (e.g., 10) to the copy
    # Step 3: Sort the copy
    # Step 4: Return the sorted copy
```

Test your solution

```python
# Test append_and_sort function
L = [3, 1, 4]
original = L.copy()
result = append_and_sort(L)
expected = [1, 3, 4, 10]
assert result == expected, f"Expected {expected}, got {result}"
assert L == original, f"Original list should be unchanged, got {L}"
print("✓ Test passed: Function works without mutating original")
print(f"Original: {L}")
print(f"Result: {result}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 19: Mutating Lists Based on Conditions

Exercise Description

Write a function `remove_elements_above(L, n)` that removes all elements greater than `n` from the list `L`. Make sure to handle list mutation properly inside a loop.

Your solution

```
def remove_elements_above(L, n):
    # Step 1: Iterate through the list safely (avoid mutation issues)
    # Step 2: Check each element against threshold n
    # Step 3: Remove elements greater than n
    # Step 4: Handle list mutation properly during iteration
```

Test your solution

```
# Test remove_elements_above function
L = [1, 5, 3, 8, 2, 7]
remove_elements_above(L, 5)
expected = [1, 5, 3, 2]
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed: Elements above threshold removed correctly")
print(f"Result: {L}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 20: Aliasing and Function Modification

Exercise Description

Create a list `a = [10, 20, 30]` and an alias `b = a`. Write a function `modify_list(L)` that appends the value `100` to `L`. Check if both `a` and `b` are modified when you pass `a` to the function.

Your solution

```
def modify_list(L):
    # Step 1: Append the value 100 to the list L
    # Step 2: Demonstrate that this modifies the original list
```

Test your solution

```
# Test modify_list function and aliasing
a = [10, 20, 30]
b = a  # Create alias
```

```
modify_list(a)
expected = [10, 20, 30, 100]
assert a == expected, f"Expected {expected}, got {a}"
assert b == expected, f"Alias should also be modified, got {b}"
print("✓ Test passed: Both original and alias modified")
print(f"a: {a}")
print(f"b: {b}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 21: String to List Conversion

Exercise Description

Convert the string `s = "Hello, world"` to a list of characters and store the result in variable `result`. Then print both the original string and the resulting list.

Your solution

```
# s = "Hello, world"
# Convert the string to a list using the list() function
# Print the original string
# Print the resulting list
```

Test your solution

```
assert result == ['H', 'e', 'l', 'l', 'o', ',', ' ', 'w', 'o', 'r', 'l', 'd']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 22: String Split by Space

Exercise Description

Split the string `s = "The brown fox is jumping around hello"` by spaces and store the result in variable `result`.

Your solution

```
# s = "The brown fox is jumping around hello"
# Split the string by spaces using the split method with space as delimiter
```

Test your solution

```
assert result == ['The', 'brown', 'fox', 'is', 'jumping', 'around', 'hello']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 23: List Join with Comma

Exercise Description

Given the list `l = ['The', 'brown', 'fox', 'is', 'jumping', 'around', 'hello']`, join the elements with ", " (comma and space) and store the result in variable `result`.

Your solution

```
# l = ['The', 'brown', 'fox', 'is', 'jumping', 'around', 'hello']
# On the string ", " use the join function passing the list l
```

Test your solution

```
assert result == "The, brown, fox, is, jumping, around, hello"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 24: String Replace Characters

Exercise Description

Given the string `s = "The brown fox is jumping around hello"`, replace all occurrences of the letter "o" with "OO" and store the result in variable `result`.

Your solution

```
# Step 1: Work with the provided input string stored in a variable.
# Step 2: Build a new string where a chosen character is consistently replaced by a chosen te
# Step 3: Save the final string into the variable named 'result'.
```

Test your solution

```
assert result == "The br0Own f0Ox is jumping ar0Ound hell0O"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 25: String Remove Characters

Exercise Description

Given the string `s = "The brown fox is jumping around hello"`, remove all occurrences of the letter "o" by replacing them with empty string and store the result in variable `result`.

Your solution

```
# Step 1: Use the given input string stored in a variable.
# Step 2: Construct a new string where every occurrence of a certain character is removed.
# Step 3: Store the final string in the variable named 'result'.
```

Test your solution

```
assert result == "The brwn fx is jumping arund hell"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 26: String List Sorting

Exercise Description

Given the list of strings `l = ["kiwi", "apple", "pear", "ananas", "Pear", "Ananas", "0nanas", "!orange"]`, sort it using the `sorted()` function and store the result in variable `result`. Note how Python sorts strings lexicographically (dictionary order).

Your solution

```
l = ["kiwi", "apple", "pear", "ananas", "Pear", "Ananas", "0nanas", "!orange"]
# call the sorted function and pass l as an argument
```

Test your solution

```
assert result == ['!orange', '0nanas', 'Ananas', 'Pear', 'ananas', 'apple', 'kiwi', 'pear']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 27: List Sorting Comparison

Exercise Description

Given two lists `ln = [4, 90, 67, 58, 13, 28]` and `lm = [4, 90, 967, 58, 13, 28]`, demonstrate the difference between `sorted()` function and `sort()` method. Store the sorted version of `ln` using `sorted()` in `result_sorted`, and sort `lm` in-place using `sort()` method. Store the final `lm` in `result_sort_method`.

Your solution

```
ln = [4, 90, 67, 58, 13, 28]
lm = [4, 90, 967, 58, 13, 28]
# Use sorted() function to create a new sorted list from ln
# Use sort() method to sort lm in-place (modifies the original list)
# Store the modified lm as result
```

Test your solution

```
assert result_sorted == [4, 13, 28, 58, 67, 90]
```

Test your solution

```
assert result_sort_method == [4, 13, 28, 58, 90, 967]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 28: List Reverse Method

Exercise Description

Given the list `lm = [4, 90, 967, 58, 13, 28]`, reverse it in-place using the `reverse()` method and store the result in variable `result`.

Your solution

```
lm = [4, 90, 967, 58, 13, 28]
# Reverse the list in-place using the reverse() method
# Store the modified list as result
```

Test your solution

```
assert result == [28, 13, 58, 967, 90, 4]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 29: List Slice Reverse

Exercise Description

Given the list `lm = [4, 90, 967, 58, 13, 28]`, create a reversed copy using slice notation `[::-1]` without modifying the original list. Store the original list in `result_original` and the reversed copy in `result_reversed`.

Your solution

```
lm = [4, 90, 967, 58, 13, 28]
# Store the original list (unchanged)
# Create a reversed copy using slice notation with step -1
```

Test your solution

```
assert result_original == [4, 90, 967, 58, 13, 28]
```

Test your solution

```
assert result_reversed == [28, 13, 58, 967, 90, 4]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 30: List Aliasing Demonstration

Exercise Description

Start with `original_list = [1,2,3,4,5]`. Modify `original_list[1] = 13`, then create an alias `l1 = original_list`. Modify `l1[0] = 89` and `original_list[2] = 113`. Store the final state of `l1` in `result_l1` and `original_list` in `result_original`. Also store whether they are equal in `result_equal`

Your solution

```
original_list = [1,2,3,4,5]
# Modify the second element (index 1) to 13
# Create an alias (both variables point to the same list object)
# Modify the first element through the alias
# Modify the third element through the original reference
# Store the final states (both should be identical due to aliasing)
result_l1 =
result_original =
# Check if they are equal (should be True)
result_equal =
```

Test your solution

```
assert result_l1 == [89, 13, 113, 4, 5]
```

Test your solution

```
assert result_original == [89, 13, 113, 4, 5]
```

Test your solution

```
assert result_equal == True
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 31: List Aliasing with Concatenation

Exercise Description

Start with `l1 = [1, 2, 3]` and create an alias `l2 = l1`. Modify `l1[0] = 99` and `l2[1] = 13`, then concatenate `l1 + l2` and store the result in variable `result`.

Your solution

```
l1 = [1, 2, 3]
# Create an alias l2 (both variables point to the same list)
# Modify the first element through l1
# Modify the second element through l2 (affects the same list)
# Concatenate the list with itself (since l1 and l2 are the same)
result =
```

Test your solution

```
assert result == [99, 13, 3, 99, 13, 3]
```

Test your solution

```
assert len(result) == 6
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 32: List Append Method

Exercise Description

Given the list `l1 = [1,2,3]`, use the `append()` method to add the number 4 to the end of the list and store the result in variable `result`.

Your solution

```
l1 = [1,2,3]
# Append the number 4 to the end of the list
# Store the modified list as result
```

Test your solution

```
assert result == [1, 2, 3, 4]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 33: List Concatenation

Exercise Description

Given two lists `l1 = [1,2,3]` and `l2 = [4,5,6]`, concatenate them using the `+` operator and store the result in variable `result`. Also store the original lists in `result_l1` and `result_l2` to verify they remain unchanged.

Your solution

```python
l1 = [1,2,3]
l2 = [4,5,6]
# Concatenate the two lists using the + operator
result =
# Store the original lists (they remain unchanged)
result_l1 =
result_l2 =
```

Test your solution

```python
assert result == [1, 2, 3, 4, 5, 6]
```

Test your solution

```python
assert result_l1 == [1, 2, 3]
```

Test your solution

```python
assert result_l2 == [4, 5, 6]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 34: List Copying with Slice

Exercise Description

Start with `l1 = [1, 2, 3]` and create a copy using slice notation `l2 = l1[:]`. Modify `l1[0] = 99` and `l2[1] = 13`, then concatenate `l1 + l2` and store the result in variable `result`.

Your solution

```
l1 = [1, 2, 3]
# Create a copy using slice notation (creates a new list) L[:]
l2 =
# Modify the first element of the original list
# Modify the second element of the copy (independent of l1)
# Concatenate the two different lists
result =
```

Test your solution

```
assert result == [99, 2, 3, 1, 13, 3]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 35: Nested List Access

Exercise Description

Given the nested list `l = [1, 2, 3, [5, [6, [7, 10]]], [8, 9]]`, access different elements and store them in variables: `result_element_4` (element at index 4), `result_element_3` (element at index 3), `result_nested_1` (element at index 1 of the nested structure at index 3), `result_nested_2` (element at index 1 of the deeper nested structure), and `result_deepest` (the deepest element: 10).

Your solution

```python
l = [1, 2, 3, [5, [6, [7, 10]]], [8, 9]]
# Access element at index 4 (the list [8, 9])
result_element_4 =
# Access element at index 3 (the nested structure [5, [6, [7, 10]]])
result_element_3 =
# Access element at index 1 of the nested structure at index 3
result_nested_1 =
# Access element at index 1 of the deeper nested structure
result_nested_2 =
# Access the deepest element (10) at index 1 of the deepest nested structure
result_deepest =
```

Test your solution

```python
assert result_element_4 == [8, 9]
```

Test your solution

```python
assert result_element_3 == [5, [6, [7, 10]]]
```

Test your solution

```python
assert result_nested_1 == [6, [7, 10]]
```

Test your solution

```python
assert result_nested_2 == [7, 10]
```

Test your solution

```
assert result_deepest == 10
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 36: String Strip Whitespace

Exercise Description

Given the string `s = "  hello   world     "` with leading and trailing whitespace, use the `strip()` method to remove the whitespace and store the result in variable `result`. Also store the original string in `result_original` for comparison.

Your solution

```
s = "  hello   world     "
```